

Java I/O Streams

Java I/O streams

- Java programs perform I/O through streams
- A stream is an abstraction that either produces or consumes information.
- A stream is linked to a physical device by the Java I/O system.
- All streams behave in the same manner, even if the actual physical (linked) devices differ.
- Thus, the same I/O classes and methods can be applied to different types of devices.

Java I/O streams

- An input stream can abstract many different kinds of input: from a disk file, a keyboard, or network socket
- Likewise, an output stream may refer to the console, a disk file, or network connection.
- Java implements streams within the class hierarchies defined in the `java.io` package

Java I/O streams

- 3 Default streams in Java
 - Standard Input stream `System.in`
 - Standard output stream `System.out`
 - Standard error stream `System.err`
- Can a user define a stream?

Byte stream & Character Stream

- Java defines two types of stream: Byte & Character
- Byte stream provide a convenient means for handling input and output of bytes. For example, when reading or writing binary data. (Any type of data, 1 byte at a time)
- Character stream provides a convenient means for handling input and output of characters. They use Unicode and, therefore, can be internationalized. (Only text data, 2 bytes at a time)
- In some cases, character streams are more efficient than byte stream.

Byte stream

- Byte stream are defined by using two class hierarchies.
- At the top are two abstract classes: **InputStream** and **OutputStream**
- Each of these abstract classes has several subclasses that handle the differences among various devices (such as disk files, network connection and even memory buffers.)

Byte stream

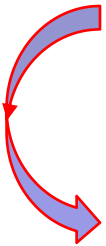
Stream Class	Meaning
BufferedInputStream	Buffered input stream
BufferedOutputStream	Buffered output stream
ByteArrayInputStream	Input stream that reads from a byte array
ByteArrayOutputStream	Output stream that writes to a byte array
DataInputStream	An input stream that contains methods for reading the Java standard data types
DataOutputStream	An output stream that contains methods for writing the Java standard data types
FileInputStream	Input stream that reads from a file
FileOutputStream	Output stream that writes to a file
FilterInputStream	Implements InputStream
FilterOutputStream	Implements OutputStream
InputStream	Abstract class that describes stream input
ObjectInputStream	Input stream for objects
ObjectOutputStream	Output stream for objects
OutputStream	Abstract class that describes stream output
PipedInputStream	Input pipe
PipedOutputStream	Output pipe
PrintStream	Output stream that contains print() and println()
PushbackInputStream	Input stream that allows bytes to be returned to the input stream.
SequenceInputStream	Input stream that is a combination of two or more input streams that will be read sequentially, one after the other

Table 13-1 The Byte Stream Classes in **java.io**

Character stream

- Character streams are defined by using two class hierarchies.
- At the top are two abstract classes: **Reader** & **Writer**
- These abstract classes handle **Unicode (16 bit)** character streams.

Character stream



Stream Class	Meaning
BufferedReader	Buffered input character stream
BufferedWriter	Buffered output character stream
CharArrayReader	Input stream that reads from a character array
CharArrayWriter	Output stream that writes to a character array
FileReader	Input stream that reads from a file
FileWriter	Output stream that writes to a file
FilterReader	Filtered reader
FilterWriter	Filtered writer
InputStreamReader	Input stream that translates bytes to characters
LineNumberReader	Input stream that counts lines
OutputStreamWriter	Output stream that translates characters to bytes
PipedReader	Input pipe
PipedWriter	Output pipe
PrintWriter	Output stream that contains print() and println()
PushbackReader	Input stream that allows characters to be returned to the input stream
Reader	Abstract class that describes character stream input
StringReader	Input stream that reads from a string
StringWriter	Output stream that writes to a string
Writer	Abstract class that describes character stream output

Abstract class, data manipulation before reading/writing data

Buffered reader, how many lines are read

Table 13-2 The Character Stream I/O Classes in **java.io**

Predefined streams

- All JAVA programs automatically import the java.lang package. This package defines a class called System, which encapsulates several aspects of the run-time environment.
- System also contains three predefined stream variables: **in, out and err.**
- These fields are declared as **public, static and final** within System
- System.out refers to the standard output stream. By default, this is the console.
- System.in refers to standard input stream, which is the keyboard by default.
- System.err refers to the standard error stream, which also is the console by default.

Reading Console Input (Character Str..)

- In Java, console input is accomplished by reading from `System.in`
- To obtain a character based stream attached to the console, wrap `System.in` in a `BufferedReader` object.

```
BufferedReader br = new BufferedReader(new  
    InputStreamReader(System.in));
```

- After this statement executes, `br` is a character based stream that is linked to the console.

Reading Characters

- To read a character from a `BufferedReader`, use `read()`.

`int read() throws IOException`

- Each time the `read()` is called, it reads a character from the input stream and returns it as an integer value.
- It returns `-1` when end of the stream is encountered.

Reading Characters

```
import java.io.*;

class BRRead {
    public static void main(String args[]) throws IOException
    {
        char c;
        BufferedReader br = new
            BufferedReader(new InputStreamReader(System.in));
        System.out.println("Enter characters, 'q' to quit.");
        // read characters
        do {
            c = (char) br.read();
            System.out.println(c);
        } while(c != 'q');
    }
}
```

Enter characters, 'q' to quit.
123abcq
1
2
3
a
b
c
q

Reading Strings

- To read a string from the keyboard, use the version of `readLine()` that is a member of the `BufferedReader` class.

```
String readLine( ) throws IOException
```

- It returns string object.

Reading Strings

```
import java.io.*;

class BRReadLines {
    public static void main(String args[]) throws
IOException
    {
        // create a BufferedReader using System.in
        BufferedReader br = new BufferedReader(new
            InputStreamReader(System.in));

        String str;
        System.out.println("Enter lines of text.");
        System.out.println("Enter 'stop' to quit.");
        do {
            str = br.readLine();
            System.out.println(str);
        } while(!str.equals("stop"));
    }
}
```

Reading Strings

```
// A tiny editor.
```

```
import java.io.*;
```

```
class TinyEdit {
```

```
    public static void main(String args[]) throws IOException  
    {
```

```
        // create a BufferedReader using System.in
```

```
        BufferedReader br = new BufferedReader(new  
                                           InputStreamReader(System.in));
```

```
        String str[] = new String[100];
```

```
        System.out.println("Enter lines of text.");
```

```
        System.out.println("Enter 'stop' to quit.");
```

```
        for(int i=0; i<100; i++) {    // Read and Store the lines
```

```
            str[i] = br.readLine();
```

```
            if(str[i].equals("stop")) break;
```

```
        }
```

```
        System.out.println("\nHere is your file:");
```

```
        // display the lines
```

```
        for(int i=0; i<100; i++) {
```

```
            if(str[i].equals("stop")) break;
```

```
            System.out.println(str[i]);
```

```
        }
```

```
    }
```

```
}
```


File Handling (byte stream)

- Two of the most often-used stream classes are: FileInputStream and FileOutputStream, which create byte streams linked to files.

`FileInputStream(String fileName)` throws `FileNotFoundException`

`FileOutputStream(String fileName)` throws `FileNotFoundException`

- `FileNotFoundException` is a subclass of `IOException`

File Handling

- To close a file, use `close()` function, which is implemented by both `FileInputStream` and `FileOutputStream`

`void close() throws IOException`

- Closing a file releases the system resources allocated to the file, allowing them to be used by another file.

File Handling - Read

- To read from a file, use `read()` function defined in `FileInputStream`.

`int read() throws IOException`

- Each time it is called, it reads a single byte from the file and returns the byte as integer value.
- `read()` returns -1 when the end of the file is encountered.

File Handling

```
import java.io.*;

class ShowFile {
    public static void main(String args[])
    {
        int i;
        FileInputStream fin;

        // First, confirm that a file name has been specified.
        if(args.length != 1) {
            System.out.println("File not found");
            return;
        }

        // Attempt to open the file.
        try {
            fin = new FileInputStream(args[0]);
        } catch(FileNotFoundException e) {
            System.out.println("Cannot Open File");
            return;
        }
    }
}
```

File Handling

```
// At this point, the file is open and can be read.
// The following reads characters until EOF is encountered.
    try {
        do {
            i = fin.read();
            if(i != -1) System.out.print((char) i);
        } while(i != -1);
    } catch(IOException e) {
        System.out.println("Error Reading File");
    }

    // Close the file.
    try {
        fin.close();
    } catch(IOException e) {
        System.out.println("Error Closing File");
    }
}
```

```
import java.io.*;
class ShowFile {
    public static void main(String args[])
    { int i;
      FileInputStream fin = null;
      if(args.length != 1) {
          System.out.println("File not found");
          return; }
      try {
          fin = new FileInputStream(args[0]);
          do {
              i = fin.read();
              if(i != -1) System.out.print((char) i);
          } while(i != -1);
      } catch(FileNotFoundException e) {
          System.out.println("File Not Found.");
      } catch(IOException e) {
          System.out.println("An I/O Error Occurred");
      } finally {
          try {
              if(fin != null) fin.close(); }
          catch(IOException e) {
              System.out.println("Error Closing File"); }
      }
    }
}
```

File Handling - Write

- To write to a file, use `write()` function defined by `FileOutputStream`.

```
void write(int byteval) throws IOException
```

- This method writes the byte specified by `byteval` to the file. Although, `byteval` is declared as an integer, only the lower eight bits are written to the file.
- If an error occurs during writing, an `IOException` is thrown.

```

import java.io.*;
class CopyFile {
    public static void main(String args[]) throws IOException
    {
        int i;
        FileInputStream fin = null;
        FileOutputStream fout = null;
        // First, confirm that both files has been specified.
        if(args.length != 2) {
            System.out.println("Usage: CopyFile from to");
            return;
        }
        finally {
            // Co
            try {
                if(fin != null) fin.close();
            } catch(IOException e2) {
                System.out.println("Error Closing Input File");
            }
            do
            try {
                if(fout != null) fout.close();
            } catch(IOException e2) {
                System.out.println("Error Closing Output File");
            }
        }
    }
}

```


File Handling

- JDK 7 added a new feature that offers another way to manage resources, such as file stream, by automating the closing process.

```
try(resource-specification) {  
    // use the resource  
}
```

- Here, resource-specification is a statement that declares and initializes a resource, such as a file stream
- When the try block ends, the resource is automatically released.

```
import java.io.*;
class ShowFile {
    public static void main(String args[])
    {
        int i;
        // First, confirm that a file name has been specified.
        if(args.length != 1) {
            System.out.println("Usage: ShowFile filename");
            return;
        }
        // The following code uses a try-with-resources statement to open a
        // file and then automatically close it when the try block ends.
        try(FileInputStream fin = new FileInputStream(args[0])) {
            do {
                i = fin.read();
                if(i != -1) System.out.print((char) i);
            } while(i != -1);
        } catch(FileNotFoundException e) {
            System.out.println("File Not Found.");
        } catch(IOException e) {
            System.out.println("An I/O Error Occurred");
        }
    }
}
```

File Handling

- The scope of the resource is limited to the try-with-resource statement.
- More than one resources can be managed by semicolon in a single try statement.

```
try (FileInputStream fin = new FileInputStream(args[0]);  
    FileOutputStream fout = new FileOutputStream(args[1]))
```

File Class

- The File class does not specify how information is retrieved from or stored in files; it describes the properties of a file itself.
- A File object is used to obtain or manipulate the information associated with a disk file, such as the permissions, time, date, and directory path, and to navigate subdirectory hierarchies.
- The following constructors can be used to create File objects:
File(String directoryPath)
File(String directoryPath, String filename)
File(File dirObj, String filename)

```
File f1 = new File("/");  
File f2 = new File("/", "autoexec.bat");  
File f3 = new File(f1, "autoexec.bat");
```

File Class

- File defines many methods that obtain the standard properties of a File object

```
import java.io.File;
class FileDemo {
    static void p(String s) {
        System.out.println(s);
    }
    public static void main(String args[]) {
        File f1 = new File("/java/COPYRIGHT");

        p("File Name: " + f1.getName());
        p("Path: " + f1.getPath());
        p("Abs Path: " + f1.getAbsolutePath());
        p("Parent: " + f1.getParent());
        p(f1.exists() ? "exists" : "does not exist");
        p(f1.canWrite() ? "is writeable" : "is not writeable");
        p(f1.canRead() ? "is readable" : "is not readable");
        p("is " + (f1.isDirectory() ? "" : "not" + " a directory"));
        p(f1.isFile() ? "is normal file" : "might be a named pipe");
        p(f1.isAbsolute() ? "is absolute" : "is not absolute");
        p("File last modified: " + f1.lastModified());
        p("File size: " + f1.length() + " Bytes");
    }
}
```

File Class

```
import java.io.File;
class FileDemo {
    static void p(String s) {
        System.out.println(s);
    }
    public static void main(String args[]) {
        File f1 = new File("/java/COPYRIGHT");

        p("File Name: " + f1.getName());
        p("Path: " + f1.getPath());
        p("Parent: " + f1.getParent());
        p(f1.exists() ? "exists" : "does not exist");
        p(f1.canWrite() ? "is writeable" : "is not writeable");
        p(f1.canRead() ? "is readable" : "is not readable");
        p("is " + (f1.isDirectory() ? "" : "not") + " a directory");
        p(f1.isFile() ? "is normal file" : "might be a named pipe");
        p(f1.isAbsolute() ? "is absolute" : "is not absolute");
        p("File last modified: " + f1.lastModified()); //milliseconds
        p("File size: " + f1.length() + " Bytes");
    }
}
```

```
File Name: COPYRIGHT
Path: /java/COPYRIGHT
Parent: /java
exists
is writeable
is readable
is not a directory
is normal file
is absolute
File last modified: 812465204000
File size: 695 Bytes
```

File Class

- File also includes two useful utility methods. The first is `renameTo( )`, shown here:

```
boolean renameTo(File newName)
```

- Here, the filename specified by `newName` becomes the new name of the invoking File object. **It will return true upon success and false if the file cannot be renamed** (if you either attempt to rename a file so that it moves from one directory to another or use an existing filename, for example).
- The second utility method is `delete( )`, which deletes the disk file represented by the path of the invoking File object. It is shown here:

```
boolean delete()
```
- You can also use `delete( )` to delete a directory if the directory is empty. **`delete( )` returns true if it deletes the file and false if the file cannot be removed.**

File Class

Method	Description
<code>void deleteOnExit()</code>	Removes the file associated with the invoking object when the Java Virtual Machine terminates.
<code>boolean isHidden()</code>	Returns true if the invoking file is hidden. Returns false otherwise.
<code>boolean setLastModified(long <i>millisec</i>)</code>	Sets the time stamp on the invoking file to that specified by <i>millisec</i> , which is the number of milliseconds from January 1, 1970, Coordinated Universal Time (UTC).
<code>boolean setReadOnly()</code>	Sets the invoking file to read-only.

Directories

- A directory is a File that contains a list of other files and directories.
- When create a File object and it is a directory, the `isDirectory( )` method will return true.
- In this case, call `list( )` on that object to extract the list of other files and directories inside.

```
String[ ] list( )
```

The list of files is returned in an array of String objects

Directories

```
import java.io.File;

class DirList {
    public static void main(String args[]) {
        String dirname = "/java";
        File f1 = new File(dirname);

        if (f1.isDirectory()) {
            System.out.println("Directory of " + dirname);
            String s[] = f1.list();

            for (int i=0; i < s.length; i++) {
                File f = new File(dirname + "/" + s[i]);
                if (f.isDirectory()) {
                    System.out.println(s[i] + " is a directory");
                } else {
                    System.out.println(s[i] + " is a file");
                }
            }
        } else {
            System.out.println(dirname + " is not a directory");
        }
    }
}
```

Directories

```
Directory of /java  
bin is a directory  
lib is a directory  
demo is a directory  
COPYRIGHT is a file  
README is a file  
index.html is a file  
include is a directory  
src.zip is a file  
.hotjava is a directory  
src is a directory
```

Directories

- limit the number of files returned by the `list( )` method to include only those files that match a certain filename pattern, or filter. To do this, use a second form of `list( )`,

```
String[ ] list(FilenameFilter FFObj)
```

- In this form, `FFObj` is an object of a class that implements the `FilenameFilter` interface.
- `FilenameFilter` defines only a single method, `accept( )`, which is called once for each file in a list. Its general form is:

```
boolean accept(File directory, String filename)
```

- The `accept( )` method returns `true` for files in the directory specified by `directory` that should be included in the list (that is, those that match the `filename` argument), and returns `false` for those files that should be excluded.

Directories

```
public class OnlyExt implements FilenameFilter {
    String ext;

    public OnlyExt(String ext) {
        this.ext = "." + ext;
    }

    public boolean accept(File dir, String name) {
        return name.endsWith(ext);
    }
}
```

Directories

```
import java.io.*;
class DirListOnly {
    public static void main(String args[]) {
        String dirname = "/java";
        File f1 = new File(dirname);
        FilenameFilter only = new OnlyExt("html");
        String s[] = f1.list(only);
        for (int i=0; i < s.length; i++) {
            System.out.println(s[i]);
        }
    }
}
```

Directories

- Another two useful File utility methods are `mkdir( )` and `makedirs( )`.
- The `mkdir( )` method creates a directory, returning **true on success and false on failure**. Failure indicates that the path specified in the File object already exists, or that the directory cannot be created because the entire path does not exist yet.
- To create a directory for which no path exists, use the `makedirs( )` method. It creates both a directory and all the parents of the directory.

Input Stream

- InputStream is an abstract class that defines Java's model of streaming byte input.
- All of the methods in this class will throw an IOException on error conditions.

Input Stream

Method	Description
<code>int available()</code>	Returns the number of bytes of input currently available for reading.
<code>void close()</code>	Closes the input source. Further read attempts will generate an IOException .
<code>void mark(int <i>numBytes</i>)</code>	Places a mark at the current point in the input stream that will remain valid until <i>numBytes</i> bytes are read.
<code>boolean markSupported()</code>	Returns true if mark() / reset() are supported by the invoking stream.
<code>int read()</code>	Returns an integer representation of the next available byte of input. -1 is returned when the end of the file is encountered.
<code>int read(byte <i>buffer</i>[])</code>	Attempts to read up to <i>buffer.length</i> bytes into <i>buffer</i> and returns the actual number of bytes that were successfully read. -1 is returned when the end of the file is encountered.
<code>int read(byte <i>buffer</i>[], int <i>offset</i>, int <i>numBytes</i>)</code>	Attempts to read up to <i>numBytes</i> bytes into <i>buffer</i> starting at <i>buffer[offset]</i> , returning the number of bytes successfully read. -1 is returned when the end of the file is encountered.

Table 17-1. The Methods Defined by InputStream

Input Stream

Method	Description
<code>void reset()</code>	Resets the input pointer to the previously set mark.
<code>long skip(long <i>numBytes</i>)</code>	Ignores (that is, skips) <i>numBytes</i> bytes of input, returning the number of bytes actually ignored.

Table 17-1. *The Methods Defined by InputStream* (continued)

OutputStream

- OutputStream is an abstract class that defines streaming byte output.
- All of the methods in this class return a void value and throw an IOException in the case of errors

Output Stream

Method	Description
<code>void close()</code>	Closes the output stream. Further write attempts will generate an IOException .
<code>void flush()</code>	Finalizes the output state so that any buffers are cleared. That is, it flushes the output buffers.
<code>void write(int <i>b</i>)</code>	Writes a single byte to an output stream. Note that the parameter is an int , which allows you to call write() with expressions without having to cast them back to byte .
<code>void write(byte <i>buffer</i>[])</code>	Writes a complete array of bytes to an output stream.
<code>void write(byte <i>buffer</i>[], int <i>offset</i>, int <i>numBytes</i>)</code>	Writes a subrange of <i>numBytes</i> bytes from the array <i>buffer</i> , beginning at <i>buffer[offset]</i> .

Table 17-2. *The Methods Defined by OutputStream*

SequenceInputStream

- The `SequenceInputStream` class allows you to concatenate multiple `InputStream`s.
- The construction of a `SequenceInputStream` is different from any other `InputStream`.
- A `SequenceInputStream` constructor uses either a pair of `InputStream`s or an `Enumeration` of `InputStream`s as its argument:

```
SequenceInputStream(InputStream first,  
InputStream second)
```

```
SequenceInputStream(Enumeration streamEnum)
```

```
import java.io.*;
import java.util.*;

class InputStreamEnumerator implements Enumeration<FileInputStream>
{
    private Enumeration<String> files;

    public InputStreamEnumerator(Vector<String> files) {
        this.files = files.elements();
    }

    public boolean hasMoreElements() {
        return files.hasMoreElements();
    }

    public FileInputStream nextElement() {
        try {
            return new FileInputStream(files.nextElement().toString());
        } catch (IOException e) {
            return null;
        }
    }
}
```

```
class SequenceInputStreamDemo {
    public static void main(String args[]) {
        int c;
        Vector<String> files = new Vector<String>();
        files.addElement("file1.txt");
        files.addElement("file2.txt");
        files.addElement("file3.txt");
        InputStreamEnumerator ise = new InputStreamEnumerator(files);
        InputStream input = new SequenceInputStream(ise);
        try {
            while ((c = input.read()) != -1)
                System.out.print((char) c);
        } catch (NullPointerException e) {
            System.out.println("Error Opening File.");
        } catch (IOException e) {
            System.out.println("I/O Error: " + e);
        } finally {
            try {
                input.close();
            } catch (IOException e) {
                System.out.println("Error Closing SequenceInputStream");
            }
        }
    }
}
```

Reader Class

- Reader is an abstract class that defines Java's model of streaming character input.
- All of the methods in this class will throw an `IOException` on error conditions.

Method	Description
<code>abstract void close()</code>	Closes the input source. Further read attempts will generate an IOException .
<code>void mark(int <i>numChars</i>)</code>	Places a mark at the current point in the input stream that will remain valid until <i>numChars</i> characters are read.
<code>boolean markSupported()</code>	Returns true if mark() / reset() are supported on this stream.
<code>int read()</code>	Returns an integer representation of the next available character from the invoking input stream. -1 is returned when the end of the file is encountered.
<code>int read(char <i>buffer</i>[])</code>	Attempts to read up to <i>buffer.length</i> characters into <i>buffer</i> and returns the actual number of characters that were successfully read. -1 is returned when the end of the file is encountered.
<code>abstract int read(char <i>buffer</i>[], int <i>offset</i>, int <i>numChars</i>)</code>	Attempts to read up to <i>numChars</i> characters into <i>buffer</i> starting at <i>buffer[offset]</i> , returning the number of characters successfully read. -1 is returned when the end of the file is encountered.
<code>boolean ready()</code>	Returns true if the next input request will not wait. Otherwise, it returns false .
<code>void reset()</code>	Resets the input pointer to the previously set mark.
<code>long skip(long <i>numChars</i>)</code>	Skips over <i>numChars</i> characters of input, returning the number of characters actually skipped.

Table 17-3. *The Methods Defined by Reader*

Writer Class

- Writer is an abstract class that defines streaming character output.
- All of the methods in this class return a void value and throw an IOException in the case of errors.

Method	Description
<code>abstract void close()</code>	Closes the output stream. Further write attempts will generate an IOException .
<code>abstract void flush()</code>	Finalizes the output state so that any buffers are cleared. That is, it flushes the output buffers.
<code>void write(int <i>ch</i>)</code>	Writes a single character to the invoking output stream. Note that the parameter is an int , which allows you to call write with expressions without having to cast them back to char .
<code>void write(char <i>buffer</i>[])</code>	Writes a complete array of characters to the invoking output stream.
<code>abstract void write(char <i>buffer</i>[], int <i>offset</i>, int <i>numChars</i>)</code>	Writes a subrange of <i>numChars</i> characters from the array <i>buffer</i> , beginning at <i>buffer[offset]</i> to the invoking output stream.
<code>void write(String <i>str</i>)</code>	Writes <i>str</i> to the invoking output stream.
<code>void write(String <i>str</i>, int <i>offset</i>, int <i>numChars</i>)</code>	Writes a subrange of <i>numChars</i> characters from the array <i>str</i> , beginning at the specified <i>offset</i> .

Table 17-4. *The Methods Defined by Writer*

Console Class

- It was added to java.io in JDK 6
- Used to read from and write to the console , if one exists.
- Console is primarily a convenience class because most of its functionality is available through System.in and System.out
- Console supplies NO constructors. Instead, a Console object is obtained as follows
 - static Console console()
 - If console is available, then a reference to it is returned. Otherwise Null is returned.

Method	Description
<code>void flush()</code>	Causes buffered output to be written physically to the console.
<code>Console format(String <i>fmtString</i>, Object...<i>args</i>)</code>	Writes <i>args</i> to the console using the format specified by <i>fmtString</i> .
<code>Console printf(String <i>fmtString</i>, Object...<i>args</i>)</code>	Writes <i>args</i> to the console using the format specified by <i>fmtString</i> .
<code>Reader reader()</code>	Returns a reference to a Reader connected to the console.
<code>String readLine()</code>	Reads and returns a string entered at the keyboard. Input stops when the user presses ENTER. If the end of the console input stream has been reached, null is returned. An IOException is thrown on failure.
<code>String readLine(String <i>fmtString</i>, Object...<i>args</i>)</code>	Displays a prompting string formatted as specified by <i>fmtString</i> and <i>args</i> , and then reads and returns a string entered at the keyboard. Input stops when the user presses ENTER. If the end of the console input stream has been reached, null is returned. An IOException is thrown on failure.
<code>char[] readPassword()</code>	Reads a string entered at the keyboard. Input stops when the user presses ENTER. The string is not displayed. If the end of the console input stream has been reached, null is returned. An IOException is thrown on failure.
<code>char[] readPassword(String <i>fmtString</i>, Object... <i>args</i>)</code>	Displays a prompting string formatted as specified by <i>fmtString</i> and <i>args</i> , and then reads a string entered at the keyboard. Input stops when the user presses ENTER. The string is not displayed. If the end of the console input stream has been reached, null is returned. An IOException is thrown on failure.
<code>PrintWriter writer()</code>	Returns a reference to a Writer connected to the console.